Compositional Concept Synthesis via CLIP-Guided Primitive Stitching

April 10, 2025

Paolo Cursi Pietro Signorino

Abstract

In this paper we explore the task of creating a composite image given a natural language prompt from a set of given primitives, such as combining "person" and "umbrella" to form "a person under an umbrella." Each primitive is represented as a visual element, and a composition function arranges them spatially to form a complete scene. To guide this composition, we use CLIP to evaluate how well the rasterized image of the composed scene aligns with a natural language description of the target concept. The optimization objective is to maximize the cosine similarity between the image embedding and the text embedding produced by CLIP.

1. Introduction

Our work focuses on the synthesis of complex composite image by stitching together a set of predefined primitives. Each target concept is associated with a collection of k primitives, which are arranged to maximize the cosine similarity between the generated image embedding and the corresponding text embedding produced by CLIP. Initially, we implemented this task using the `pgpelib` library, where each concept was directly mapped to its corresponding primitives.

Early experiments revealed that our initial optimization methods were insufficient, leading us to explore alternative strategies. We discovered that the Covariance Matrix Adaptation (CMA) algorithm performed significantly better, improving the quality of the synthesized images in terms of their semantic alignment with the target descriptions.

Despite the improvements provided by CMA, the performance of our method was hindered by a challenging loss function landscape. The loss, defined as the negative cosine similarity, exhibited many local maxima, which impeded the optimization process from converging to the global optimum. To address this, we modified the loss function by incorporating an additional component designed to smooth the landscape. This adjustment aimed to mitigate the effect of local maxima, thereby facilitating more robust convergence.

While this enhanced approach yielded promising results, it also introduced certain limitations that will be discussed in detail in the following sections.

Email: Paolo Cursi <cursi.2155622@studenti.uniroma1.it>, Pietro Signorino <signorino.2149741@studenti.uniroma1.it>.

Deep Learning and Applied AI 2024, Sapienza University of Rome, 2nd semester a.y. 2023/2024.

2. Related Work

We based our work on the paper *Modern Evolution Strategies for Creativity: Fitting Concrete Images and Abstract Concepts* (Tian & Ha, 2022) (ES-CLIP).

ES-CLIP presents a method that leverages evolutionary strategies to optimize image generation based on CLIP's joint embedding space. The so called *population* is a set of triangles, each one represented by the position of the three vertices relative to a given canvas and a color. These features are optimized to maximize the cosine similarity between the CLIP embedding of the triangles rendered on the canvas and the CLIP embedding of a target concept.

Inspired by this work, we want to do something similar by composing visual scenes from a set of semantic primitives, instead of triangles.

3. Method

In order to stitch together k primitives to form a scene, we need to define a few components: the scene configuration and the optimization strategy.

3.1. Scene Configuration & Pipeline

We represent a scene configuration as a matrix of shape $(k, 5)$ and each entry is bounded to be in the $[0, 1]$ range. Each row corresponds to one primitive and the five parameters describe its spatial attributes.

- **x, y**: the position of the top-right corner of the primitive with respect to the canvas size;

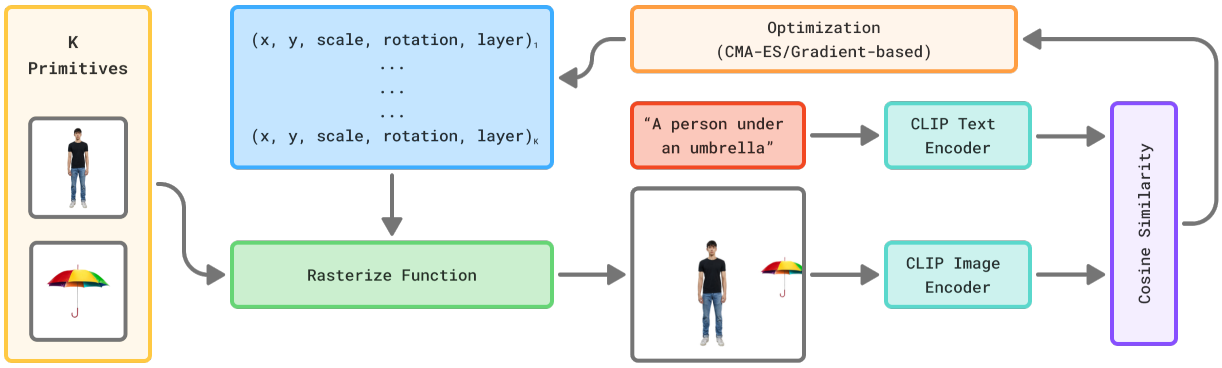

Figure 1. The whole pipeline of our method. We want the k (in this case 2) primitives shown in the *yellow* box, to be rasterized to have a similar CLIP embedding as the prompt, shown in the *red* box.

- **scale**: the relative size of the primitive (i.e. a percentage of the full primitive size);
- **rotation**: the angle of rotation, $0 - 365$ degrees normalized;
- **layer**: the rendering order (i.e., which primitives appear in front). We order the rasterization process, such that primitives with smaller layer parameter will be rasterized first, hence be in the back.

We created a `rasterize` function, which uses these parameters and the primitives to compose the scene and rasterizes it into an image. This image is then fed into CLIP's image encoder to compute its embedding. The cosine similarity between this image embedding and the text embedding of the target concept serves as the objective function. An overview of this pipeline is shown in Figure 1.

3.2. CMA-ES Optimization

To find a good configuration of the primitives, we first adopted a gradient-free optimization strategy using *Covariance Matrix Adaptation Evolution Strategy* (Hansen & Ostermeier, 1996) (CMA-ES). This algorithm iteratively samples configurations, evaluates them using the cosine similarity score, and updates a multivariate Gaussian distribution to bias future samples toward more promising regions in the search space.

3.3. Gradient-Based Approach

In parallel, we also experimented with a gradient-based optimization method. We created a new rasterization function which is differentiable and has some limitations: rotating the primitives was pretty hard, so we decided to implment it only if a preliminary test would have been promising. This function allowed us to compute gradients of the cosine similarity loss with respect to the primitive parameters and apply gradient descent to improve the layout.

3.4. Loss Function Exploration

After many experiments we found out that both methods often became stuck in poor local optima, so we decided to explore the loss function's landscape. This eventually led us to explore modifications to the loss in an effort to change the optimization surface, as detailed in the next section.

4. Results

In this section we will report the results of each method individually, then some common improvements we applied to both of them and finally we will show the loss exploration we talked about in the previous section.

4.1. Default approach

This approach is the one we implemented first, using the `CMA-ES` python library. For the gradient-based approach we just implemented the `differentiable_rasterize` function (which operates on PyTorch tensors) and used the `Adam` optimizer to update the parameters of the primitives. This yielded poor results as shown in the figure.

4.2. Loss Exploration

After these poor results with both methods, we decided to explore the loss function landscape. We found that the loss function was very noisy and had many local optima, which made it difficult for the optimization algorithms to converge to a good solution. An example is show in the figure. In order to plot the contour plot of the loss function, we choose a simple example "A person under un umbrella", that has two primitives: *person* and *umbrella*. We embedded every combination of $(x, y, scale)$ parameters with values $0.3, 0.38, 0.46, 0.54, 0.62, 0.7$ for both primitives, while keeping the rotation and layer fixed. The loss function was computed for each combination and plotted in

a contour plot. Of course, close points in this space are not guaranteed to be close in the original space, but still this is useful because it shows us how the majority of parameters combination yield the same cosine similarity.

4.3. Loss Improvements

5. Discussion and Conclusions

TODO

References

Hansen, N. and Ostermeier, A. Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation. In *Proceedings of IEEE international conference on evolutionary computation*, pp. 312–317. IEEE, 1996.

Tian, Y. and Ha, D. Modern evolution strategies for creativity: Fitting concrete images and abstract concepts, 2022. URL https://arxiv.org/abs/2109.08857.